How to become an Arborist: An introductory guide

Bao Minh Tran – s224236373

SIT221 – Data Structures and Algorithms
Deakin University

May 10, 2026

Summary of content

Technique	**Question answered**
Euler Tour	How can a subtree become an array interval?
LCA	Where do two tree paths first meet?
HLD	How can path queries be reduced to a few ranges?
Centroid Decomposition	How can global tree problems be split recursively?

Prerequisites

Graph and tree

A graph is $G = (V, E)$. A tree is an undirected, connected, acyclic graph [1], [2].

- Trees with n vertices have exactly $n - 1$ edges.
- There is a unique path between any two vertices.

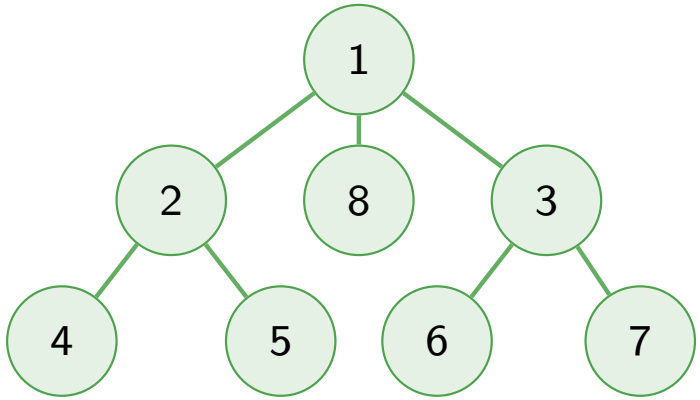
Rooted tree setup

A rooted tree at node 1 is discussed, represented by an adjacency list.

- parent/child relation
- depth of a node
- subtree rooted at u

Why trees are powerful

- General graphs can have many paths, cycles, and repeated choices.
- Trees remove that ambiguity: every path is unique.
- Many questions become about intervals, ancestors, and balanced components.
- This is why preprocessing can make later queries much faster.



RMQ and Segment Tree

For an array of n elements, denoted as $arr = [arr_0, \cdots, arr_{n-1}]$. Suppose having a query, namely $\mathcal{Q}(l, r) = f([arr_l, \cdots, arr_r])$, where $0 \leq l \leq r \leq n-1$. The array arr can be reconstructed as a Segment Tree subject to $f(\cdot)$.

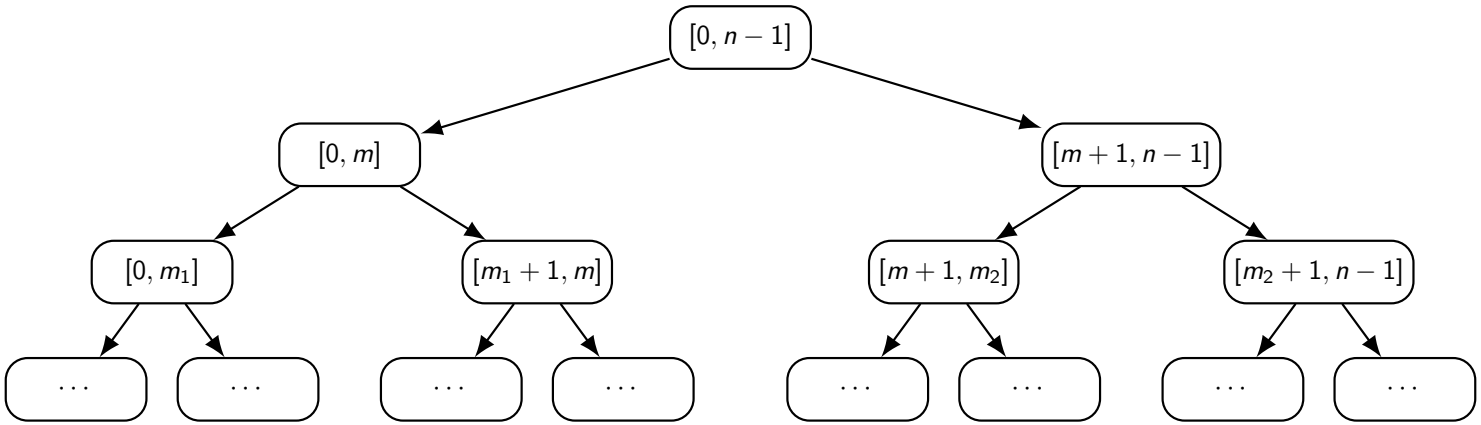


Figure: General structure of a segment tree.

A generic Segment Tree is organised as follows:

$$ST[l][r] = \begin{cases} arr[l], & l = r \\ f\left(ST\left[l\right]\left[\left\lfloor \frac{l+r}{2} \right\rfloor\right], ST\left[\left\lfloor \frac{l+r}{2} \right\rfloor + 1\right]\left[r\right]\right), & l < r \end{cases}$$

RMQ and Sparse Table

For an array of n elements, denoted as $arr = [arr_0, \cdots, arr_{n-1}]$. Suppose having a query, namely $\mathcal{Q}(l, r) = f([arr_l, \cdots, arr_r])$, where $0 \leq l \leq r \leq n-1$. A Sparse Table, namely $st[][]$, can be constructed from arr by the utilisation of the recursion relation as follows:

$$st[j][i] = \begin{cases} arr[i] & j = 0 \\ st[j-1][i] \ \cup \ st[j-1][i+2^{j-1}] & j > 0 \end{cases}$$

Therefore, $st[j][i] = f([i \cdots i + 2^j - 1])$. In other words, $st[j][i]$ answers the query of an interval starting at i^{th} with length of $(i + 2^j - 1) - i + 1 = 2^j$. Consequently, a query $\mathcal{Q}(l, r)$ is returned in $\mathcal{O}(1)$ by:

$$\mathcal{Q}(l, r) = f\left(\left[l \cdots l + (2^k - 1)\right]\right) \bigcup f\left(\left[r - 2^k + 1 \cdots (r - 2^k + 1) + (2^k - 1)\right]\right)$$

where $k \doteq \lfloor \log_2(r - l + 1) \rfloor$.

Euler Tour: core idea

Problem

Given a tree $T = (V, E)$. Find the sum of nodes in a subtree rooted at $v \in V$.

- Perform a DFS from the root.
- Record `tin[u]` when first entering u.
- Record `tout[u]` after all descendants of u are processed.
- Store visited vertices in an Euler array.

Important result

Every subtree becomes one contiguous interval:

$$S_u \Rightarrow [\texttt{tin[u]}, \texttt{tout[u]})$$

Complexity

DFS visits every vertex and edge once: $\mathcal{O}(n)$ for a tree.

Euler Tour: what it gives us

Subtree size

$$|S_u| = \texttt{tout[u]} - \texttt{tin[u]}$$

Ancestor test

u is an ancestor of v when:

$$\texttt{tin[u]} \leq \texttt{tin[v]} < \texttt{tout[u]}$$

Presenter point

The tree is still a tree, but the query is now phrased as an array problem.

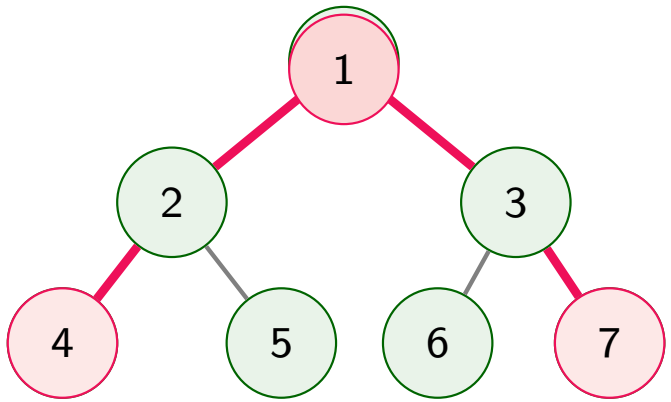
- Subtree queries become range queries.
- This allows segment trees, Fenwick trees, or prefix sums to be used.
- Euler Tour is also a building block for LCA.

LCA: path meeting point

Definition

$LCA(u, v)$ is the deepest node that is an ancestor of both u and v.

- It identifies where two root-to-node paths meet.
- It is useful for distances and path decomposition.
- It avoids walking the full path for every query.



$LCA(4, 7) = 1$

LCA: turns into RMQ

- LCA can be transformed into a Range Minimum Query.
- During Euler Tour, store:
  - `euler[]`: visited nodes
  - `first[]`: first occurrence of each node
  - `depth[]`: depth at each tour position
- Between `first[u]` and `first[v]`, the minimum-depth node is the LCA.

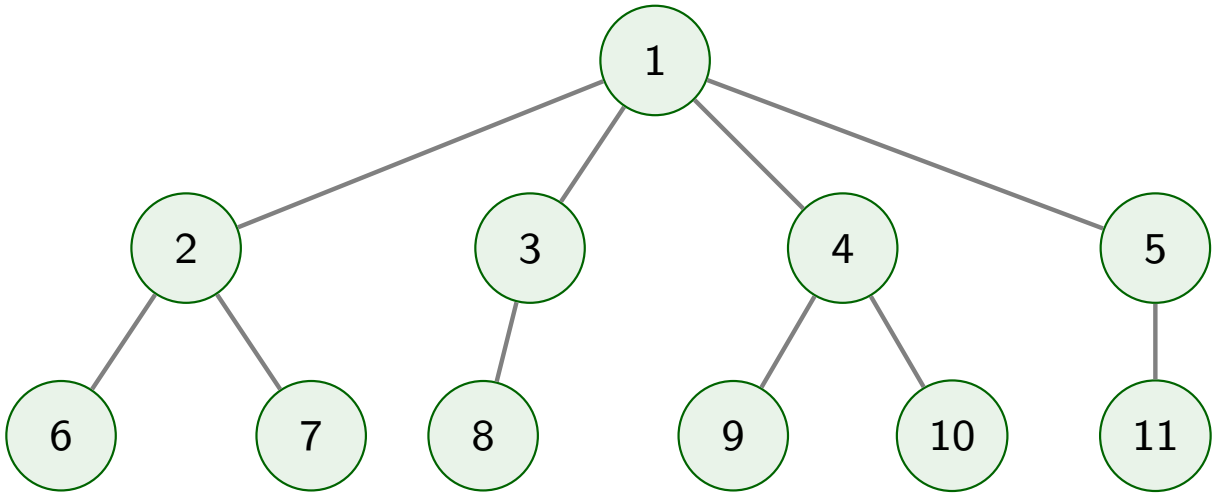
Method	Build	Query
Segment Tree	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$
Sparse Table	$\mathcal{O}(n \log n)$	$\mathcal{O}(1)$

Highlights

Euler Tour + RMQ is the bridge between tree ancestry and array range queries.

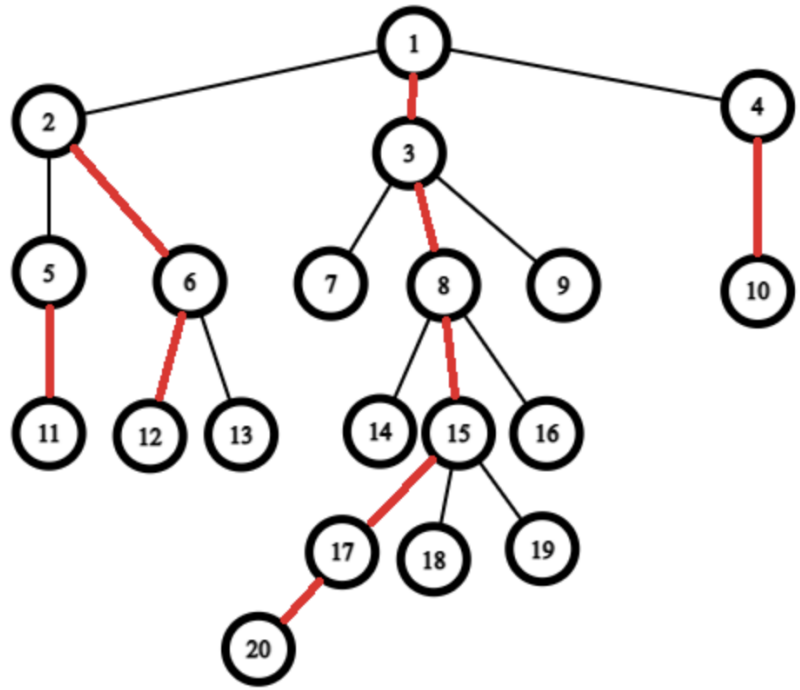
HLD: A problem statement

Given a tree $T = (V, E)$. For two vertices $u, v \in V$, compute the sum of nodes along the path from u to v.



HLD: splitting a path into chains

- For each node, choose the child with the largest subtree as the heavy child.
- Heavy edges join into heavy paths.
- Light edges connect different heavy paths.
- Any path crosses only $\mathcal{O}(\log n)$ heavy paths.



Red edges represent heavy edges.

Why it works

Every time we cross a light edge downward, the remaining subtree size drops by at least about half.

HLD: query workflow

1. DFS computes parent and subtree size.
2. Decomposition assigns each node to a chain.
3. Nodes in each heavy path are placed consecutively in an array.
4. Build a segment tree over that array.

Path query

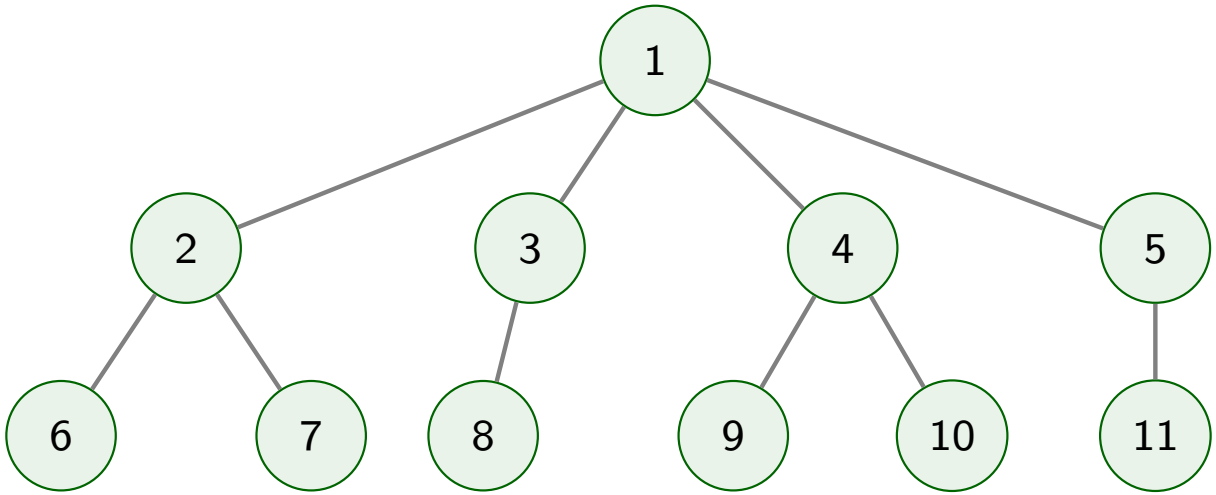
Move from u and v chain-by-chain until both are on the same chain, combining segment tree answers.

Operation	Cost
Build	$\mathcal{O}(n)$
Vertex update	$\mathcal{O}(\log n)$
Path query	$\mathcal{O}(\log^2 n)$

Best fit: online path sums, maximum edge weights, and changing vertex values [3], [4].

Centroid Decomposition: A problem statement

Given a tree $T = (V, E)$. Count the number of paths with exactly k vertices in T.

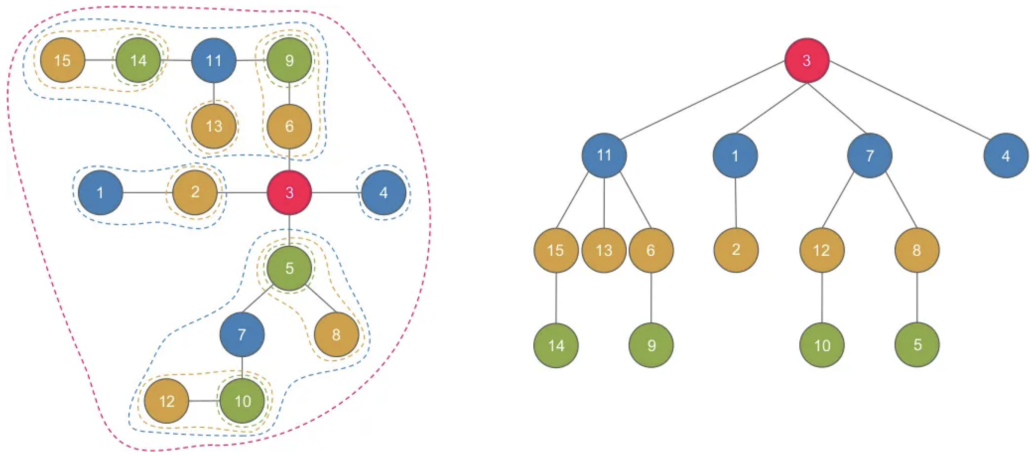


Centroid: balanced separator

Centroid

A node whose removal leaves every remaining component with at most half of the original nodes.

- Every tree has one or two centroids.
- If there are two, they are adjacent.
- Removing a centroid creates balanced subproblems.



Original tree (left) and Centroid tree (right).

Centroid vs center: a centroid balances component sizes; a center minimizes maximum distance.

Centroid Decomposition: workflow

1. Compute component sizes.
2. Find a centroid of the current component.
3. Process information through that centroid.
4. Mark it removed.
5. Recurse on each remaining component.

Complexity

Each level processes nodes in its components, and the centroid tree has $\mathcal{O}(\log n)$ levels:

$$\mathcal{O}(n \log n)$$

- Count paths of length exactly k.
- Maintain nearest marked node.
- Aggregate distance-based values.

Choosing the right tool

Tool	Core idea	Use it when
Euler Tour	Subtrees become intervals.	The query is about a subtree or ancestry.
LCA	Two paths meet at one deepest ancestor.	The query needs path identity or distance.
HLD	A path becomes a small number of array ranges.	Path values change or need aggregation.
Centroid Decomp.	Balanced recursive separators.	The query is global and distance-based.

Main takeaway

These techniques are related, but not interchangeable. First identify the shape of the query, then choose the structure that makes that shape simple.

Conclusion and practice

- Euler Tour converts hierarchy into intervals.
- LCA turns path identity into an ancestry/RMQ problem.
- HLD supports online path queries and updates.
- Centroid Decomposition handles global distance/path problems.

Closing sentence

The shared pattern is to exploit tree structure first, then apply the right array or decomposition data structure.

DSA playgrounds

- `https://wiki.vnoi.info`
- `https://cp-algorithms.com`
- `https://usaco.guide`
- `https://codeforces.com`
- `https://cses.fi/problemset/`

References

[1] R. Diestel, *Graph Theory*, 5th ed. Berlin, Germany: Springer, 2017, ISBN: 9783662536216. DOI: 10.1007/978-3-662-53622-3.

[2] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data Structures and Algorithms in Java*, 6th ed. Wiley, 2014.

[3] VNOI Wiki, *Vnoi wiki*, https://wiki.vnoi.info, Accessed: 2026-05-05, 2024.

[4] CP-Algorithms, *Cp-algorithms*, https://cp-algorithms.com, Accessed: 2026-05-05, 2024.